

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem Solving Task

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA15

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.*
(BL4)

Assessment Tool Weightage: 20%

Dave's Taxonomy: Articulation (Level 4)

Total Marks: 40

Question Count: 5

Code of Conduct

I ___ G.JAAHNAVI(192525277) ___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the student: ___ G.Jaahnavi ___

Assessment Tasks Industry Problem Solving Task

1. An e-commerce company receives clickstream, transaction, and review data from millions of users daily. Propose a big data analytics pipeline and explain how the 5Vs influence architecture design.
2. A healthcare organization collects structured patient records, semi-structured insurance claims, and unstructured medical images. Design an approach to manage and secure this data ecosystem.
3. A telecom company wants to detect churn patterns using high-volume customer interaction data. Identify the big data challenges and recommend suitable analytics and integration strategies.
4. A smart city project generates sensor streams from traffic, air quality, and surveillance systems. Explain how distributed file systems and data integration tools can support large-scale analysis.
5. A financial analytics firm must comply with strict privacy requirements while processing high-velocity transaction data. Design a secure big data solution and justify the controls chosen.

Industry Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Industry Context	20%	Demonstrates strong understanding of real-world problem context	Good understanding	Basic understanding	Weak understanding
Application of Big Data Concepts	25%	Applies highly relevant big data ideas and tools	Mostly appropriate application	Partial application	Weak application
Solution Design	25%	Solution is practical, scalable, and well reaso	Reasonable solution	Basic solution with gaps	Unclear solution
Security / Risk Awareness	15%	Strong awareness of security and operational	Reasonable awareness	Limited awareness	Poor awareness

Clarity and Professionalism	15%	Well-structured and professional response	Generally clear	Moderately clear	Poorly presented
-----------------------------	-----	---	-----------------	------------------	------------------

1. E-Commerce Big Data Analytics Pipeline and 5Vs

Problem Understanding

An e-commerce company receives **clickstream data, transaction data, and customer review data from millions of users every day**. This data is generated continuously and exists in different formats. The company needs a scalable Big Data architecture to collect, store, process, analyze, and visualize the information.

The main objective is to understand customer behavior, improve recommendations, identify purchasing patterns, detect fraudulent transactions, and improve business decisions.

Proposed Big Data Analytics Pipeline

The proposed pipeline can be represented as:

Data Sources → Data Ingestion → Data Storage → Data Processing → Analytics/ML → Visualization → Business Decisions

1. Data Sources

The major data sources include:

- Website and mobile application clickstreams
- Product searches and page views
- Customer transactions
- Payment information
- Product reviews and ratings
- Customer interaction logs
- Social media feedback

The data can be both structured and unstructured.

2. Data Ingestion

A distributed data ingestion system should collect data continuously.

For example:

- **Apache Kafka** for real-time streaming
- Batch ingestion for historical transaction and review data
- APIs for external data sources

Kafka can handle large volumes of incoming events and distribute them to multiple processing systems.

3. Data Storage

A scalable distributed storage system such as **Hadoop Distributed File System (HDFS)** or cloud object storage can be used.

Data can be organized into:

- **Raw/Data Lake layer** – original incoming data
- **Processed layer** – cleaned and transformed data
- **Curated layer** – analytics-ready data

This approach prevents the loss of original information while allowing efficient analysis.

4. Data Processing

Processing can be performed using **Apache Spark**.

Spark can perform:

- Data cleaning
- Transformation
- Aggregation

Both batch and streaming processing can be supported.

5. Analytics and Machine Learning

Machine learning models can analyze customer behavior.

Examples include:

- Recommendation systems
- Customer segmentation
- Fraud detection

6. Visualization

Tools such as dashboards and BI platforms can present:

- Sales trends
- Customer behavior
- Popular products

This allows managers to make data-driven decisions.

Influence of the 5Vs

5V	Meaning	Influence on Architecture
Volume	Millions of users generate huge amounts of data	Requires distributed and scalable storage
Velocity	Data arrives continuously and rapidly	Requires real-time ingestion and stream processing

Variety	Data has different formats	Requires flexible storage and processing systems
Veracity	Data may contain errors or inconsistencies	Requires validation, cleaning and quality controls
Value	Data must provide useful business benefits	Requires analytics and ML systems

Security Considerations

Since customer and transaction information is sensitive, the system should use:

- Encryption during storage and transmission
- Authentication and authorization
- Role-Based Access Control

Conclusion

A distributed Big Data pipeline using **Kafka, scalable storage, Spark, machine learning and visualization tools** can efficiently process e-commerce data. The 5Vs directly influence storage, processing, scalability, data quality and analytics design. Such an architecture enables the company to convert massive customer data into useful business insights.

2. Healthcare Data Management and Security

Problem Understanding

A healthcare organization handles three major types of information:

1. **Structured patient records**
2. **Semi-structured insurance claims**
3. **Unstructured medical images**

These datasets differ in format, size, access requirements and sensitivity. Healthcare data is highly confidential, so the system must provide both **efficient data management and strong security**.

Proposed Architecture

Patient Records + Insurance Claims + Medical Images



Data Ingestion



Data Lake / Distributed Storage



Data Processing & Integration



Analytics



Healthcare Applications

1. Structured Patient Records

Structured information includes:

- Patient ID
- Age
- Medical history

A relational database can be used for transactional healthcare applications.

2. Semi-Structured Insurance Claims

Insurance claims may contain formats such as:

- JSON
- XML
- Electronic healthcare records
- Claim documents

A flexible NoSQL or data-lake-based system can store such data.

3. Unstructured Medical Images

Medical images such as:

- X-rays
- CT scans
- MRI images
- Ultrasound images

require high-capacity storage. Distributed storage or cloud object storage can provide scalable storage.

Data Integration

A common data platform should integrate all three data types.

An **ETL/ELT pipeline** can perform:

- Extraction
- Transformation
- Cleaning

Metadata management should also be used to identify where data came from and how it has been processed.

Security Measures

Healthcare data requires strong security controls.

1. Encryption

Data should be encrypted:

- **At rest**
- **In transit**

This protects information from unauthorized access.

2. Authentication

Only authorized users should be allowed to access healthcare systems.

Multi-factor authentication can provide additional protection.

3. Role-Based Access Control

Different users should have different permissions.

For example:

- Doctors → patient clinical information
- Insurance staff → claims information
- Administrators → operational information
- Analysts → controlled/anonymized datasets

4. Data Masking and Anonymization

Personally identifiable information should be protected when data is used for analytics.

For example, direct identifiers can be removed or masked before researchers access datasets.

5. Audit Logging

Every important access and modification should be recorded.

This helps identify:

- Unauthorized access
- Suspicious activity
- Data misuse

6. Backup and Disaster Recovery

Healthcare data must be protected against:

- Hardware failures
- Cyberattacks
- Accidental deletion
- System failures

Regular backups and disaster recovery mechanisms should therefore be implemented.

Analytics Applications

Once properly integrated and secured, the data can support:

- Disease prediction
- Patient risk analysis
- Medical image analysis

Conclusion

A healthcare Big Data ecosystem should combine **distributed storage, data integration, scalable processing and strong security controls**. Separating raw, processed and analytics-ready data improves manageability, while encryption, access control, anonymization, auditing and backup mechanisms protect sensitive patient information.

3. Telecom Customer Churn Detection

Problem Understanding

A telecom company generates enormous amounts of customer interaction data every day. The data may include:

- Call records
- SMS activity
- Internet usage
- Recharge history
- Customer complaints
- Service requests
- Billing information
- Network usage
- Customer support interactions

The objective is to identify customers who are likely to leave the service, known as **churn prediction**.

Major Big Data Challenges

1. Volume

Millions of customers generate huge amounts of call, network and interaction records.

Solution: Use distributed storage and parallel processing.

2. Velocity

Telecom data is generated continuously and requires rapid processing.

Solution: Use real-time streaming technologies such as Kafka and stream-processing systems.

3. Variety

Data comes from multiple systems and formats.

Examples:

- Structured billing records
- Semi-structured logs
- Text-based complaints

Solution: Use data integration and flexible data storage.

4. Veracity

Customer information may contain:

- Missing values
- Duplicate records
- Incorrect entries
- Inconsistent formats

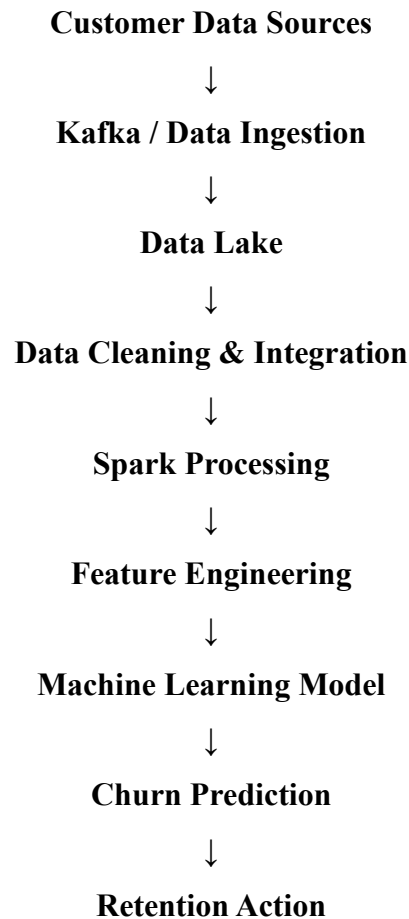
Solution: Data validation, cleaning and quality checks.

5. Scalability

The system must continue to perform when the number of customers and events increases.

Solution: Distributed cloud infrastructure and horizontal scaling.

Proposed Churn Analytics Architecture



Data Integration Strategy

Data from different systems should be integrated using a centralized data platform.

Customer information can be connected using a common identifier such as:

Customer ID

This allows the organization to combine:

- Usage behavior

- Billing history
- Complaints
- Network quality
- Customer service interactions

Analytics Strategy

Historical customer data can be used to train machine learning models.

Potential techniques include:

- Logistic Regression
- Decision Trees
- Random Forest

The model can classify customers as:

Low Churn Risk / Medium Churn Risk / High Churn Risk

Feature Engineering

Useful features include:

- Average monthly usage
- Number of complaints
- Recharge frequency

These features help identify changes in customer behavior.

Real-Time Churn Detection

Streaming analytics can identify sudden behavioral changes.

For example:

Customer usage decreases + complaints increase + recharge frequency decreases → High churn probability

The company can then provide appropriate retention offers or customer support.

Security and Privacy

Customer information should be protected using:

- Encryption
- Authentication
- Access control

Conclusion

A combination of **distributed Big Data storage, real-time ingestion, data integration, Spark processing and machine learning** can help telecom companies detect churn patterns. Early prediction allows the company to take preventive action, improve customer satisfaction and reduce customer loss.

4. Smart City Data Using Distributed File Systems and Integration Tools

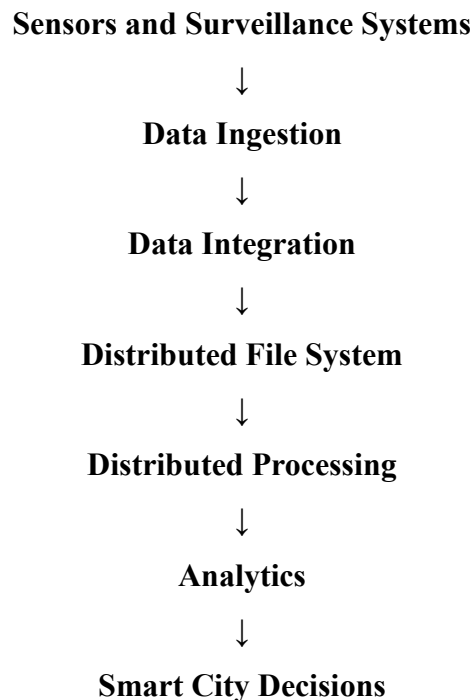
Problem Understanding

A smart city generates continuous data from:

- Traffic sensors
- Air-quality sensors
- Surveillance systems
- Weather sensors

This creates large-scale, high-velocity and heterogeneous data. A scalable Big Data architecture is required to store and analyze this information.

Proposed Architecture



Role of Distributed File Systems

A distributed file system such as **HDFS** divides large datasets into blocks and stores them across multiple machines.

Advantages

1. Scalability

Additional storage nodes can be added when data increases.

2. Fault Tolerance

Data can be replicated across multiple nodes so that hardware failure does not result in complete data loss.

3. Large-Scale Storage

It can store large volumes of sensor readings, logs and multimedia data.

4. Parallel Processing

Data stored across different nodes can be processed in parallel, improving performance.

Role of Data Integration Tools

Smart city data comes from different sources and formats. Integration tools help combine these datasets.

For example:

- Traffic sensor data
- Air-quality readings
- Surveillance metadata
- Public transport information

can be integrated using common timestamps, locations or sensor IDs.

Real-Time Data Integration

Streaming systems can continuously collect sensor events.

For example:

Traffic sensor → Data ingestion → Stream processing → Traffic dashboard

If congestion is detected, authorities can respond quickly.

Data Processing

Distributed processing tools such as Apache Spark can perform:

- Traffic pattern analysis
- Pollution analysis

Example

Suppose traffic sensors show increasing vehicle density at a particular junction.

The system can combine:

- Current traffic data
- Historical traffic patterns
- Weather conditions

The analytics system can then identify congestion and help authorities optimize traffic signals or routes.

Security Considerations

Smart city infrastructure should protect:

- Sensor data
- Surveillance information
- Citizen information

Security measures include:

- Encryption

- Authentication
- Access control

Conclusion

Distributed file systems provide **scalable, reliable and fault-tolerant storage**, while data integration tools combine information from different smart-city sources. Together with distributed processing and real-time analytics, they enable efficient large-scale analysis and support faster, data-driven city management.

5. Secure Big Data Solution for Financial Transactions

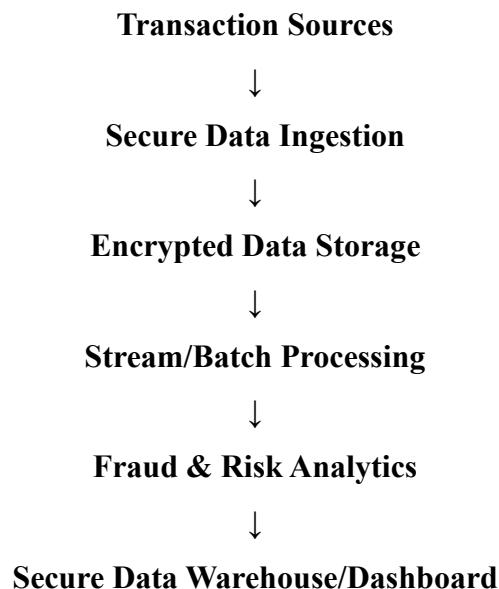
Problem Understanding

A financial analytics firm processes **high-velocity transaction data** while being required to follow strict privacy requirements. Financial information is highly sensitive, so the architecture must provide both **high performance and strong security**.

The system should support:

- Real-time transaction processing
- Fraud detection
- Risk analysis

Proposed Secure Architecture



1. Secure Data Ingestion

A streaming platform such as Kafka can receive transactions at high speed.

The communication channel should be protected using encryption and authenticated connections.

2. Data Storage

A scalable data lake or distributed storage system can store transaction information.

Data should be divided into:

- Raw data
- Processed data
- Curated/analytics data

Access should be controlled at every layer.

3. Encryption

Encryption is one of the most important controls.

Data at Rest

Stored transaction information should be encrypted.

Data in Transit

Information moving between systems should use secure encrypted communication.

This reduces the risk of unauthorized interception.

4. Identity and Access Management

Only authorized employees and applications should access financial data.

Role-Based Access Control (RBAC) can be implemented.

For example:

- Data analyst → limited analytical data
- Security officer → security logs
- Administrator → controlled system management
- Application → only required data

This follows the principle of **least privilege**.

5. Data Masking and Tokenization

Sensitive financial information should not be exposed unnecessarily.

Sensitive values can be:

- Masked
- Tokenized
- Anonymized

This reduces the impact of unauthorized access.

6. Real-Time Fraud Detection

High-velocity transaction data can be processed using stream analytics.

For example, the system can identify unusual patterns such as:

- Multiple transactions in a short period
- Unusual transaction locations
- Sudden changes in transaction behavior

fraud detection model can assign a risk score to each transaction.

7. Monitoring and Audit Logs

All important system activities should be monitored.

Audit logs can record:

- User access
- Data modifications
- Failed login attempts

These logs support investigation and compliance requirements.

8. Backup and Disaster Recovery

Financial data must remain available even during failures.

The solution should include:

- Regular backups
- Replicated storage
- Disaster recovery

9. Data Governance

A governance framework should define:

- Who owns the data
- Who can access it
- How it should be classified
- How sensitive information should be handled

Security Controls and Justification

Security Control	Purpose
Encryption	Protects data from unauthorized disclosure
Authentication	Verifies user identity
RBAC	Restricts access according to user role
Least Privilege	Gives users only required permissions
Data Masking	Hides sensitive information
Tokenization	Protects sensitive financial identifiers
Audit Logging	Tracks user and system activities
Monitoring	Detects suspicious behavior
Backup	Prevents permanent data loss
Disaster Recovery	Maintains business continuity

Conclusion: A secure financial Big Data architecture should combine **high-speed data ingestion, distributed processing, encryption, access control, masking/tokenization, real-time fraud detection.**